ShopStream

Data Pipeline System Design Document

Batch and streaming architecture for retail analytics, operational metrics, fraud detection, and governed data products.

Revision: dbt replaces Dataform as the transformation framework. BigQuery remains the production warehouse target, while DuckDB provides an optional, credential-free local validation target.

Component	Role	Status
Transformation	dbt models, snapshots, tests, and adapter macros	Implemented
Local validation	dbt with embedded DuckDB and deterministic seeds	Implemented
Production target	dbt with BigQuery Bronze, Silver, and Gold datasets	Compiled
Orchestration	Apache Airflow project packaged with Astronomer	Implemented

Version 2.0 | November 2026

1. Overview

ShopStream is a production-minded retail data platform designed to analyze user behavior, improve conversion rates, support daily business intelligence, and provide low-latency signals for fraud and operational use cases. The design uses separate batch and streaming paths because those workloads have different latency, replay, and serving requirements.

The repository is intentionally local-first. Core transformations can be built and tested without a cloud warehouse, while the same dbt project can compile for BigQuery. Managed GCP services remain deployment targets rather than prerequisites for every development cycle.

1.1 Objectives

- Ingest clickstream, transaction, product catalog, and customer support data.
- Produce authoritative Silver and Gold analytical models on a batch cadence.
- Process selected events continuously for operational and fraud-detection use cases.
- Support replay, testing, observability, governance, and controlled cloud costs.
- Demonstrate portable engineering practices through Apache Beam, dbt, and open local tooling.

1.2 Architectural style

ShopStream follows a Lambda-style architecture. Pub/Sub is the shared event entry point. One branch lands raw data for batch transformations; the other processes events continuously. Streaming outputs are optimized for speed, while batch outputs are the reconciled source for historical reporting.

The branches are intentionally independent after Pub/Sub. They must use stable event identifiers and documented reconciliation rules so low-latency results can later be compared with authoritative batch results.

1.3 Current implementation versus target architecture

Component	Role	Status
Source simulator	Publishes four local source files to Pub/Sub topics	Implemented
Batch landing	Pub/Sub consumer writes date-partitioned files to Cloud Storage	Implemented
dbt	Builds customer, product, sales, and behavior models	Implemented
DuckDB	Runs the complete dbt build locally from deterministic fixtures	Implemented
Beam/Dataflow	Writes generic streaming envelopes to BigQuery	Implemented
DLP/Dataplex/ML	Governance and advanced-consumption capabilities	Planned

2. Architecture Diagram

The diagram shows the GCP deployment view. dbt owns SQL transformations across the Bronze, Silver, and Gold warehouse layers. DuckDB is not shown because it is a local validation target rather than a production serving component.

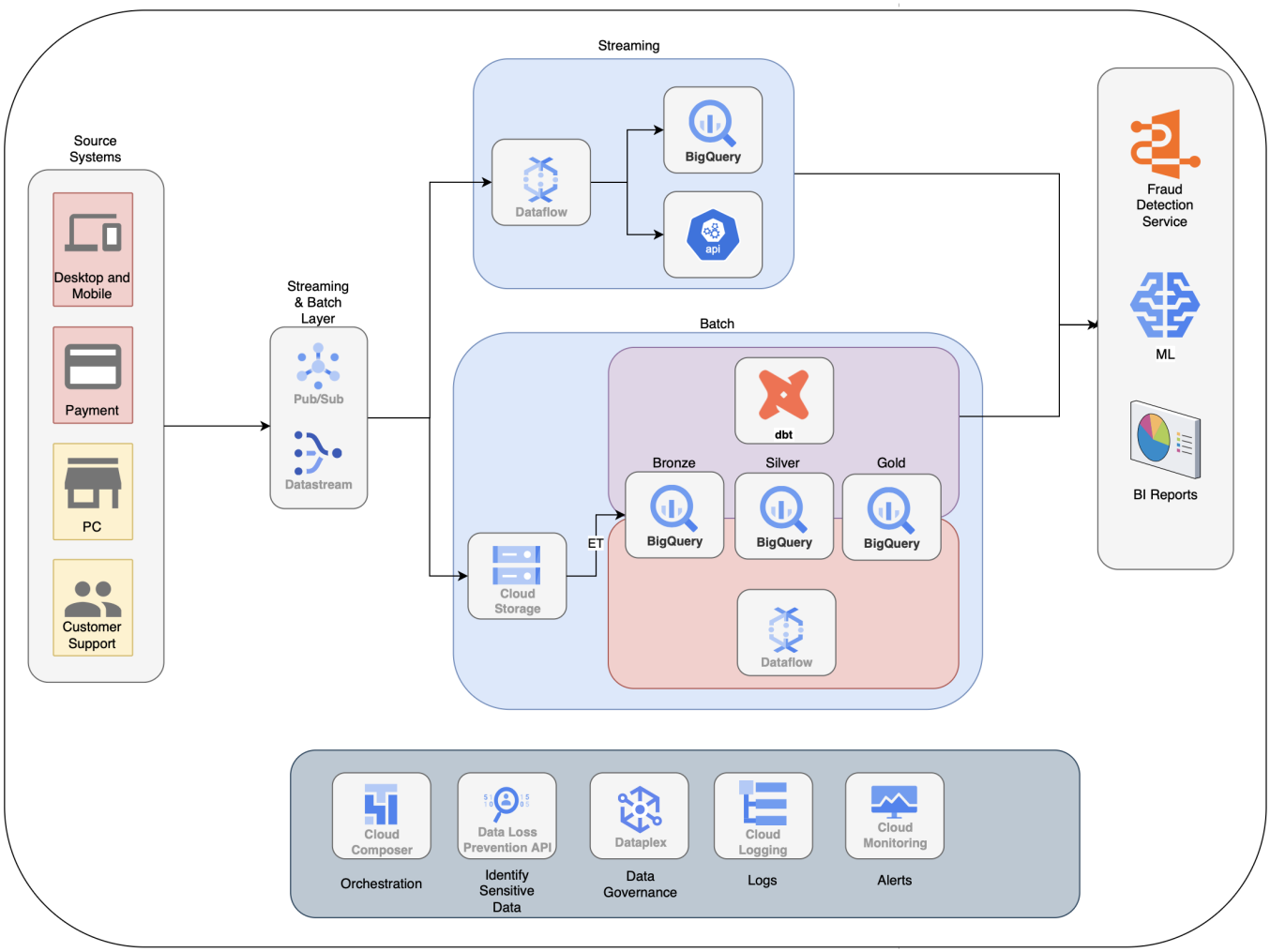


Figure 1. ShopStream batch and streaming architecture. Services in the supporting platform band represent the target operating model; some remain planned.

3. Architecture Explanation

3.1 Data sources and ingestion

- Desktop and mobile applications generate clickstream events.
- The payment system produces order transactions.
- The product catalog supplies product and inventory updates.
- Customer support produces ticket and interaction data.

Pub/Sub acts as the central asynchronous ingestion point and decouples producers from downstream consumers. The repository simulates these sources with four local files and four topics. Datastream is an optional future CDC path for transactional databases; it is not required by the current implementation.

3.2 Streaming layer

Apache Beam defines the portable streaming transformations. The DirectRunner supports small local tests and debugging, while DataflowRunner is the managed GCP target. One pipeline instance is submitted per topic.

- Parse Pub/Sub payloads while retaining message metadata and source topic.
- Attach publish time and ingestion time for latency and replay analysis.
- Write generic raw envelopes to topic-specific BigQuery streaming tables.
- Future transforms can add windowed metrics, sessionization, anomaly detection, and dead-letter routing.

3.3 Batch layer and the dbt migration

The batch path archives Pub/Sub messages in date-partitioned Cloud Storage paths. BigQuery external tables expose those files as the Bronze layer. dbt then builds and tests the Silver and Gold models.

dbt replaces Dataform. The migration removes the Dataform SQLX project and Node dependencies, converts table declarations to dbt sources, converts assertions to dbt data tests, and isolates warehouse-specific JSON behavior in adapter-dispatched macros.

- Bronze: raw GCS-backed external tables in `shopstream_bronze`.
- Silver: `d_customers`, `d_products`, and `f_sales` in `shopstream_silver`.
- Gold: `user_behavior_summary` in `shopstream_gold`.
- Customer history: dbt snapshot provides SCD Type 2 validity periods.

3.4 Local and production dbt targets

Component	Role	Status
DuckDB	Embedded local database; seeds, snapshots, models, and tests	Local only
BigQuery	GCS-backed Bronze sources and managed Silver/Gold datasets	Production

DuckDB is not a replacement for the production warehouse. It is a fast local boundary for validating model dependencies, SQL logic, snapshots, and data tests without cloud credentials. BigQuery compilation remains a separate compatibility check.

3.5 Storage and serving

- Cloud Storage is the durable raw archive and replay source for the batch path.
- BigQuery Bronze exposes raw files; Silver provides cleaned and conformed data; Gold provides report-ready aggregates.
- Streaming BigQuery tables support low-latency operational queries and future fraud signals.
- BI tools and data science workloads should primarily consume governed Silver and Gold models.

3.6 Orchestration

Apache Airflow orchestrates the batch path. The repository includes an Astronomer project for local Airflow packaging. The weekly DAG calls source ingestion, lands data in raw storage, runs the customer snapshot, builds Silver, and then builds Gold. Local Astro currently skips the BigQuery dbt tasks unless an appropriate target and credentials are configured.

- Current local orchestrator: Astronomer/Astro project.
- Potential managed targets: Astronomer deployment or Cloud Composer.
- Production Cloud Run calls require identity-token authentication.

3.7 Supporting services

- Cloud Logging and Monitoring: centralized logs, metrics, dashboards, and alerts.
- Dataplex: planned discovery, metadata, lineage, and governance capabilities.
- Sensitive Data Protection: planned scanning, classification, and redaction of PII.
- Fraud and ML services: planned consumers of streaming signals and governed analytical features.

4. Design Decisions

4.1 Batch and streaming separation

The batch layer favors completeness, replayability, complex SQL transformation, and cost-efficient historical processing. The streaming layer favors seconds-to-minutes latency for operational decisions. Streaming results are provisional; reconciled batch models are authoritative for historical reporting.

4.2 Late-arriving data

- Streaming pipelines use event time, windowing, allowed lateness, and explicit late-data handling.
- Records beyond the grace period are routed to a dead-letter or replay path.
- Batch models reprocess affected partitions and remain idempotent under repeated execution.

4.3 Schema evolution

- Every topic should carry a versioned contract and stable event identifier.
- New fields are optional by default to preserve backward compatibility.
- Breaking type changes require a new version and controlled consumer migration.
- Parquet or Avro is preferred for scalable archival; current sample inputs include JSON and CSV.

4.4 Data retention and privacy

- Raw data retention supports replay and audit, with lifecycle transitions to colder storage.
- Silver retention supports historical analysis; Gold retains compact long-term aggregates.
- Deletion requests require traceable removal or masking across raw and analytical layers.
- Sensitive fields require classification, least privilege, and logging controls.

4.5 Failure recovery

- Acknowledge Pub/Sub messages only after the destination write succeeds.
- Expect redelivery and deduplicate using stable keys.
- Use retries for transient failures and dead-letter queues for non-retryable records.
- Use Cloud Storage replay plus dbt idempotency for batch recovery.
- Use runner-managed checkpointing for stateful streaming workloads.

4.6 Portability and cloud dependency

Portability is achieved by separating processing logic from execution adapters. dbt models target DuckDB locally and BigQuery in production. Beam transforms are portable, while Pub/Sub and BigQuery I/O remain GCP-specific adapters. DirectRunner is a development tool, not a production streaming runtime.

4.7 Scaling strategy

- Pub/Sub provides elastic ingestion and independent consumer subscriptions.
- Dataflow autoscaling handles distributed streaming growth in the managed deployment.
- BigQuery partitioning and clustering reduce scans as data volume grows.
- dbt model selection, incremental patterns, and partition-aware SQL bound batch work.
- Cloud Storage lifecycle rules control raw-data retention cost.

4.8 Data quality and reconciliation

- dbt tests enforce non-null, uniqueness, and relationship expectations.
- Additional freshness, volume, accepted-value, and referential-integrity checks are planned.
- Event IDs, source timestamps, and ingestion timestamps support cross-path reconciliation.
- Daily batch results can correct provisional streaming aggregates.

4.9 Security

- Use least-privilege service accounts for ingestion, transformation, and orchestration.
- Prefer Workload Identity Federation over long-lived service-account JSON keys.
- Keep source data, credentials, local databases, and environment files out of Git.
- Authenticate Airflow calls to private Cloud Run endpoints with identity tokens.

5. Capacity Planning Example

The following values are illustrative, not measured ShopStream production traffic. Load tests and real event-size distributions are required before provisioning decisions.

5.1 Ingestion

- Peak assumption: 1,000,000 events per minute, or approximately 16,667 events per second.
- At 1 KB per event, inbound payload throughput is approximately 16.7 MB per second before protocol overhead.
- Topic partitioning, publisher quotas, subscriber concurrency, and backlog age must be monitored.

5.2 Streaming processing

- Worker requirements depend on parsing cost, windowing, state, enrichment, and sink behavior.
- Begin with bounded load tests, measure throughput and latency, then tune worker type and autoscaling limits.
- Track end-to-end event latency, watermark delay, backlog age, error rate, and cost per million events.

5.3 Storage

- At 1.44 billion 1 KB events per day, uncompressed volume is approximately 1.44 TB per day.
- Annual uncompressed volume is approximately 526 TB; compression and columnar formats materially reduce stored size.
- Retention tiers should separate raw replay data, conformed history, and compact reporting aggregates.

5.4 Query performance

- Partition by event or business date and cluster on common filter keys such as user or product.
- Use pre-aggregated Gold models or materialized views for repeated BI workloads.
- Do not use a large historical warehouse scan for sub-second fraud decisions; consume streaming features or a dedicated serving API.

5.5 Cost controls

- Set BigQuery scan limits, budgets, labels, and partition filters.
- Tune Dataflow worker bounds and avoid unnecessary shuffles.
- Use lifecycle policies for raw files and measure Composer or Astronomer deployment costs before selection.
- Keep local dbt/DuckDB validation in CI to avoid cloud spend for routine transformation tests.

6. Production Readiness

6.1 Deployment strategy

- Define Pub/Sub, buckets, datasets, service accounts, Dataflow jobs, and orchestration configuration with Terraform.
- Run dbt parse and a complete DuckDB build on pull requests.
- Compile dbt for BigQuery and perform cost-bounded integration tests before production promotion.
- Deploy Beam changes only after transform tests pass.
- Promote development to staging and then production with explicit approvals and rollback plans.

6.2 CI/CD after the dbt migration

GitHub Actions validates the dbt project without cloud credentials by loading deterministic Bronze seeds into DuckDB, running the customer snapshot, building all models, and executing data tests. A separate workflow tests and submits Beam/Dataflow jobs. Function workflows deploy source ingestion and Pub/Sub-to-GCS components.

- The repository no longer requires Dataform packages, `package.json`, or committed `node_modules`.
- dbt dependencies are pinned for reproducible local and Airflow image builds.
- BigQuery execution remains an authenticated production boundary, not something inferred from local success.

6.3 Monitoring and alerting

- Collect structured logs with correlation IDs across ingestion, Beam, Airflow, and dbt.
- Alert on Pub/Sub backlog, Dataflow failure, DAG failure, dbt test failure, freshness breach, and cost anomaly.
- Track p50/p95 latency, throughput, error rate, retry volume, late-data rate, and warehouse scan cost.
- Maintain runbooks and ownership for every production alert.

6.4 Disaster recovery

- Retain raw data so downstream tables can be rebuilt.
- Version code, contracts, orchestration, and infrastructure in Git.
- Use BigQuery time travel and controlled backups for accidental table changes.
- Test regional recovery assumptions; do not rely on managed-service availability claims alone.

6.5 Performance optimization

- Select only required columns and enforce partition filters in BigQuery.
- Use incremental or partition-scoped dbt models where full rebuilds become expensive.
- Minimize Beam shuffles and select windowing and triggers based on latency requirements.
- Compact small files before analytical scans.
- Profile each stage before optimizing: ingestion, transformation, query, and serving.

6.6 Definition of done

An architecture capability is production-ready only when implementation, tests, deployment automation, documentation, security controls, observability, and a boundary-level validation agree. Configuration alone is not proof that a managed integration works.

7. Local Validation Workflow

The dbt transformation layer can be validated without GCP. DuckDB runs in-process and writes a local database file that is excluded from version control.

Component	Role	Status
1. Setup	Install pinned dbt Core, DuckDB, and BigQuery adapters	`make dbt_setup`
2. Seed	Load deterministic Bronze fixtures into DuckDB	`dbt seed`
3. Snapshot	Create customer SCD Type 2 history	`dbt snapshot`
4. Build	Build Silver and Gold and run 13 tests	`make dbt_local`
5. Compile	Compile warehouse-specific SQL for BigQuery	`make dbt_compile_bigquery`

The local build validates transformation logic, lineage, snapshots, and tests. It does not prove BigQuery runtime permissions, external-table availability, Cloud Run authentication, or full Airflow integration. Those boundaries require separate authenticated checks.

8. Open Decisions and Next Steps

- Define the real-time serving tables and business consumers for the streaming branch.
- Specify reconciliation rules between provisional streaming metrics and authoritative batch models.
- Connect local Airflow ingestion output to the local dbt target for a true end-to-end Astro run.
- Bring BigQuery external-table DDL under infrastructure as code or a repeatable deployment workflow.
- Add versioned topic contracts, dead-letter handling, replay procedures, and schema validation.
- Add freshness, volume, accepted-value, and cross-layer reconciliation tests.
- Implement Workload Identity Federation and authenticated Airflow-to-Cloud-Run calls.
- Choose and validate the managed Airflow deployment target.
- Standardize GCP regions where practical and document data-location constraints.
- Measure real volumes and replace illustrative capacity assumptions with evidence.

9. Migration Summary

Component	Role	Status
Removed	Dataform project configuration, SQLX models, Node package files, and tracked dependencies	Complete
Added	dbt sources, models, snapshots, tests, profiles, and adapter macros	Complete
Local target	DuckDB build using deterministic Bronze fixtures	Validated
Cloud target	BigQuery compilation for production datasets	Validated
Airflow	dbt snapshot, Silver build, and Gold build tasks	Implemented
Documentation	Architecture source, exported diagram, and system design document	Updated

The result is a local-first transformation workflow that preserves BigQuery production support and reduces the need for cloud infrastructure during everyday development and CI.